

Caminos Más Cortos con Fuente Única: Comparación entre Dijkstra, Bellman-Ford y el Algoritmo Determinista $O(m \log^{2/3} n)$

José Manuel Bustinza Quispe

Ingeniería Informática y de Sistemas

UNSAAC

224867@unsaac.edu.pe

Amilcar Estacio Medrano

Ingeniería Informática y de Sistemas

UNSAAC

200822@unsaac.edu.pe

Brenda Lucía Mayhuire Chacón

Ingeniería Informática y de Sistemas

UNSAAC

231445@unsaac.edu.pe

Mirco Sair Salcedo Ataulluco

Ingeniería Informática y de Sistemas

UNSAAC

200886@unsaac.edu.pe

Resumen—El problema de caminos más cortos con fuente única (Single Source Shortest Path, SSSP) constituye uno de los problemas fundamentales en la teoría de grafos y en la ciencia de la computación, con aplicaciones directas en sistemas de navegación, ruteo de redes, logística y análisis de grafos a gran escala. Desde su introducción en 1959, el algoritmo de Dijkstra ha sido considerado el estándar clásico para resolver este problema en grafos con pesos no negativos, alcanzando una complejidad temporal de $O(m + n \log n)$ mediante estructuras de datos avanzadas.

Recientemente, se han propuesto nuevos algoritmos deterministas que logran superar este límite histórico en grafos dispersos, alcanzando una complejidad de $O(m \log^{2/3} n)$ en el modelo de comparación y suma. Esta primera entrega presenta una revisión sistemática de la literatura científica sobre el problema SSSP, abarcando al menos dieciséis artículos científicos relevantes, junto con un resumen conceptual del nuevo algoritmo $O(m \log^{2/3} n)$ y una comparación inicial frente al algoritmo de Dijkstra en términos de fundamentos teóricos, complejidad computacional y condiciones de aplicabilidad.

Index Terms—Caminos más cortos, Algoritmo de Dijkstra, Algoritmos de grafos, Complejidad computacional, Single Source Shortest Path, Estado del arte.

I. INTRODUCCIÓN

El estudio de algoritmos para el cálculo de caminos más cortos en grafos ha sido un área central de investigación en la ciencia de la computación durante más de seis décadas. En particular, el problema de caminos más cortos con fuente única (SSSP) busca determinar la distancia mínima desde un vértice origen hacia todos los demás vértices de un grafo ponderado, asumiendo pesos no negativos en sus aristas.

Desde su publicación en 1959, el algoritmo de Dijkstra se ha consolidado como la solución clásica y más utilizada para este problema, debido a su eficiencia, simplicidad conceptual y robustez práctica [1]. Mediante el uso de estructuras de datos como montículos de Fibonacci, el algoritmo alcanza una complejidad de $O(m + n \log n)$, lo que lo hace adecuado para una amplia variedad de aplicaciones reales.

Sin embargo, el crecimiento exponencial del tamaño de los grafos modernos, especialmente en contextos como redes sociales, sistemas de transporte globales y análisis de grandes volúmenes de datos, ha motivado la búsqueda de algoritmos más eficientes desde el punto de vista teórico. Durante décadas, se consideró que la complejidad de Dijkstra representaba un límite óptimo en el modelo de comparación y suma.

En los últimos años, investigaciones recientes han logrado romper este paradigma mediante el desarrollo de algoritmos deterministas que alcanzan una complejidad subóptima de $O(m \log^{2/3} n)$ para grafos dispersos [13]. Este avance representa el primer progreso determinista significativo en el problema SSSP en más de veinte años y abre nuevas perspectivas para la investigación en algoritmos de grafos.

La presente entrega corresponde a la fase inicial del proyecto y tiene como propósito construir un estado del arte sólido sobre el problema SSSP, sintetizando los aportes históricos más relevantes, las técnicas algorítmicas desarrolladas y los avances recientes que han redefinido los límites teóricos del problema.

II. OBJETIVOS DEL PROYECTO

II-A. Objetivo General

Analizar comparativamente el algoritmo de Dijkstra y los algoritmos deterministas recientes para el problema de caminos más cortos con fuente única, evaluando sus fundamentos teóricos, complejidad computacional y condiciones de aplicabilidad a partir de una revisión sistemática de la literatura científica.

II-B. Objetivos Específicos de la Entrega N°1

- Revisar al menos dieciséis artículos científicos representativos sobre el problema SSSP y áreas afines.
- Describir los fundamentos teóricos del algoritmo de Dijkstra y su evolución histórica.
- Resumir los principios y técnicas clave del nuevo algoritmo determinista con complejidad $O(m \log^{2/3} n)$.

- Realizar una comparación inicial entre el algoritmo de Dijkstra y el nuevo algoritmo en términos de complejidad temporal y condiciones de aplicabilidad.
- Identificar las líneas de investigación más prometedoras para etapas posteriores del proyecto.

III. ESTADO DEL ARTE

El problema de caminos más cortos ha sido ampliamente estudiado desde mediados del siglo XX, dando lugar a una extensa literatura que abarca enfoques clásicos, mejoras basadas en estructuras de datos y avances recientes de carácter teórico. Esta sección sintetiza los aportes más significativos identificados en la revisión bibliográfica.

III-A. Aportes Clásicos y Fundacionales

El algoritmo de Dijkstra [1] estableció las bases del estudio del problema SSSP en grafos con pesos no negativos, convirtiéndose en el estándar de referencia durante décadas por su elegancia conceptual y su eficiencia práctica. De forma paralela, algoritmos como Bellman-Ford [2], [3] ampliaron el alcance del problema al permitir pesos negativos en las aristas, a costa de un mayor costo computacional. Para el problema relacionado de todos los pares de caminos más cortos, Floyd [4] propuso una solución basada en programación dinámica con complejidad $O(n^3)$, que continúa siendo relevante en grafos densos de tamaño moderado.

III-B. Mejoras Basadas en Estructuras de Datos

Un avance decisivo en la eficiencia del algoritmo de Dijkstra fue la introducción de los montículos de Fibonacci por Fredman y Tarjan [5], los cuales permitieron reducir el costo amortizado de las operaciones de actualización de prioridad y alcanzar la complejidad $O(m + n \log n)$. Posteriormente, Thorup [6] demostró que, en grafos no dirigidos con pesos enteros, era posible resolver el problema SSSP en tiempo lineal $O(m)$, un resultado considerado óptimo bajo dichas restricciones. Pettie y Ramachandran [7] extendieron estas ideas a grafos no dirigidos con pesos reales, ofreciendo algoritmos más eficientes en escenarios específicos.

III-C. Técnicas de Escalado y Aproximación

Goldberg [8] introdujo técnicas de escalado para el problema de caminos más cortos, aprovechando la estructura numérica de los pesos para reducir el trabajo computacional. Más recientemente, Goldberg, Kennedy y Rao [9] publicaron una revisión exhaustiva sobre distancias y caminos más cortos en grafos, consolidando el panorama contemporáneo del área. En paralelo, Madry [11] desarrolló esquemas de aproximación más rápidos para el problema de flujo multicommodity fraccional, cuyas ideas técnicas han inspirado avances posteriores en algoritmos de caminos más cortos.

III-D. Caminos Más Cortos Dinámicos y Distribuidos

Durante la última década, la investigación se ha orientado hacia grafos de gran escala y entornos dinámicos o distribuidos. Henzinger et al. [10] establecieron resultados unificados de dureza para problemas dinámicos, marcando límites inferiores

importantes para el área. Nanongkai [12] propuso algoritmos distribuidos de aproximación para caminos más cortos ponderados, abordando los desafíos propios de los modelos CONGEST y similares. Chen y Wang [16] aportaron nuevos algoritmos y resultados de dureza para el problema incremental de caminos más cortos con fuente única.

III-E. Algoritmos Sublineales y Oráculos de Distancia

En la frontera actual del área, Dubois y Silva [14] propusieron un algoritmo sublineal para caminos más cortos aproximados en redes de gran tamaño, mientras que Williams y Zhang [15] desarrollaron oráculos de distancia casi óptimos para grafos planares. Estos trabajos amplían el repertorio de herramientas disponibles para el análisis y procesamiento eficiente de grafos masivos.

III-F. Punto de Inflexión: Complejidad Subóptima Determinista

En este contexto, el trabajo de Henzinger et al. [13] representa un punto de inflexión al introducir el primer algoritmo determinista que supera el límite clásico de Dijkstra en grafos dispersos, alcanzando una complejidad de $O(m \log^{2/3} n)$. Este resultado constituye el primer avance determinista significativo para el problema SSSP en el modelo de comparación y suma en más de dos décadas, y abre nuevas líneas de investigación orientadas a la optimización práctica y al diseño de algoritmos híbridos para grafos masivos.

III-G. Síntesis de la Revisión Bibliográfica

La revisión sistemática realizada en esta entrega abarca dieciséis artículos científicos representativos, distribuidos entre aportes clásicos, mejoras estructurales, técnicas de escalado, algoritmos dinámicos y distribuidos, y los más recientes avances en complejidad subóptima determinista. Esta base bibliográfica proporciona el sustento teórico necesario para las siguientes etapas del proyecto, donde se profundizará en el análisis comparativo y, eventualmente, en la evaluación experimental de los algoritmos estudiados.

IV. FUNDAMENTOS TEÓRICOS

IV-A. Problema de Caminos Más Cortos con Fuente Única

El problema de caminos más cortos con fuente única (Single Source Shortest Path, SSSP) consiste en determinar la distancia mínima desde un vértice origen s hacia todos los demás vértices de un grafo ponderado $G = (V, E)$, donde cada arista $e \in E$ tiene asociado un peso no negativo. Formalmente, se busca calcular una función de distancia $d : V \rightarrow \mathbb{R}^+$ tal que $d(v)$ represente el costo mínimo de un camino desde s hasta v .

Este problema es fundamental en teoría de grafos y aparece en numerosas aplicaciones prácticas, incluyendo ruteo de redes, navegación GPS, planificación logística y análisis de redes complejas [8], [9].

IV-B. Modelo Computacional y Complejidad

El análisis de los algoritmos de caminos más cortos se realiza comúnmente bajo el modelo de comparación y suma, en el cual las operaciones básicas permitidas son comparaciones entre valores y sumas de pesos. En este modelo, la complejidad temporal de un algoritmo se expresa en función del número de vértices $n = |V|$ y el número de aristas $m = |E|$.

Durante décadas, se consideró que el algoritmo de Dijkstra alcanzaba un límite óptimo para el problema SSSP en este modelo, particularmente en grafos dispersos donde $m \approx n$. Los avances recientes han cuestionado esta percepción, mostrando que es posible obtener mejoras asintóticas mediante técnicas algorítmicas más sofisticadas.

V. ALGORITMO DE DIJKSTRA

El algoritmo de Dijkstra, propuesto por Edsger W. Dijkstra en 1959 [1], resuelve el problema SSSP en grafos con pesos no negativos mediante un enfoque voraz. El algoritmo mantiene un conjunto de vértices cuya distancia mínima desde la fuente ya ha sido determinada y selecciona iterativamente el vértice con la menor distancia tentativa, relajando posteriormente las aristas incidentes para actualizar las distancias de sus vecinos.

Cuando se implementa utilizando una cola de prioridad basada en montículos de Fibonacci [5], el algoritmo alcanza una complejidad temporal de:

$$T_{\text{Dijkstra}}(n, m) = O(m + n \log n) \quad (1)$$

lo que lo hace eficiente para una amplia variedad de aplicaciones prácticas. La principal limitación de este enfoque radica en la necesidad de ordenar globalmente los vértices según sus distancias tentativas, lo que introduce el factor logarítmico en su complejidad temporal y constituye precisamente el cuello de botella que los nuevos algoritmos buscan superar.

VI. AVANCES RECIENTES EN ALGORITMOS SSSP

Investigaciones recientes han demostrado que es posible superar el límite clásico de Dijkstra en grafos dispersos mediante el uso de técnicas más sofisticadas de descomposición y procesamiento jerárquico del grafo. En particular, Henzinger et al. [13] proponen un algoritmo determinista que alcanza una complejidad temporal de:

$$T_{\text{Nuevo}}(n, m) = O(m \log^{2/3} n) \quad (2)$$

Este resultado constituye el primer avance determinista significativo para el problema SSSP en el modelo de comparación y suma en más de dos décadas.

VI-A. Ideas Clave del Nuevo Algoritmo

El nuevo enfoque se basa en una combinación de técnicas avanzadas, entre las cuales destacan:

- **Descomposición jerárquica del grafo:** el grafo se divide en múltiples niveles de granularidad, permitiendo resolver versiones aproximadas del problema antes de refinar la solución final.

- **Evitar el ordenamiento global:** a diferencia de Dijkstra, el algoritmo evita ordenar todos los vértices por distancia, realizando actualizaciones locales en regiones específicas del grafo.
- **Uso de aproximaciones multi-escala:** se combinan soluciones de diferentes niveles de precisión para reducir el trabajo redundante.
- **Estructuras de datos especializadas:** se emplean estructuras dinámicas que permiten procesar múltiples actualizaciones de manera eficiente.

VI-B. Comparación Conceptual con Dijkstra

Mientras que Dijkstra sigue un enfoque secuencial basado en la selección del vértice más cercano en cada iteración, el nuevo algoritmo adopta una estrategia global basada en aproximaciones progresivas. Esta diferencia permite reducir el número de operaciones de ordenamiento y, en consecuencia, mejorar la complejidad asintótica en grafos dispersos.

No obstante, esta mejora teórica viene acompañada de una mayor complejidad en la implementación y de constantes ocultas más elevadas, lo que limita su aplicabilidad práctica inmediata.

VII. COMPARACIÓN INICIAL

La comparación entre algoritmos para el problema de caminos más cortos con fuente única se realiza considerando tanto su complejidad teórica como sus condiciones de aplicabilidad. En esta entrega se realiza una comparación inicial enfocada en complejidad asintótica y restricciones; en entregas posteriores se profundizará con análisis experimental.

Desde el punto de vista asintótico, Dijkstra presenta una complejidad $O(m + n \log n)$, mientras que el nuevo algoritmo reduce el factor logarítmico en grafos dispersos. Sin embargo, esta mejora teórica no implica necesariamente un mejor rendimiento práctico en todos los escenarios, debido a las constantes ocultas y a la complejidad de implementación del nuevo enfoque.

VII-A. Comparación de Complejidades

El cuadro I resume la complejidad temporal de los algoritmos más representativos para el problema SSSP identificados en la revisión bibliográfica.

Tabla I
COMPARACIÓN DE COMPLEJIDADES DE ALGORITMOS SSSP

Algoritmo	Complejidad temporal	Restricciones
Bellman-Ford [2]	$O(nm)$	Permite pesos negativos
Dijkstra [1]	$O(m + n \log n)$	Pesos no negativos
Thorup [6]	$O(m)$	No dirigido, pesos enteros
Henzinger et al. [13]	$O(m \log^{2/3} n)$	Grafo general

Esta comparación evidencia que el nuevo algoritmo representa una mejora teórica clara sobre Dijkstra en grafos dispersos,

aunque bajo supuestos más complejos y con un costo de implementación significativamente mayor. La validación de estas ventajas teóricas mediante experimentación controlada constituirá uno de los objetivos de las entregas posteriores del proyecto.

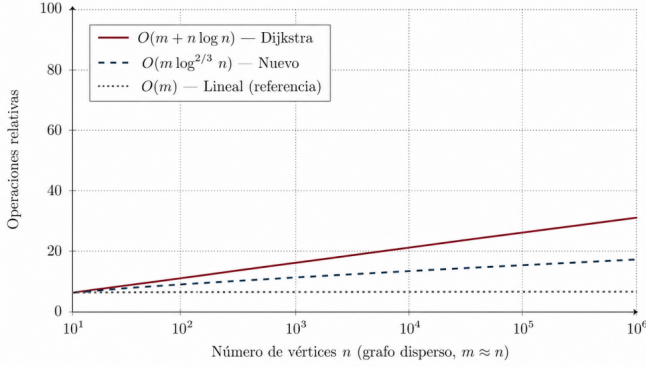


Figura 1. Operaciones relativas en función del número de vértices para grafos dispersos ($m \approx n$). A mayor tamaño del grafo, la brecha entre Dijkstra y el nuevo algoritmo se amplía.

VIII. CONCLUSIONES PRELIMINARES

La revisión bibliográfica realizada en esta primera entrega permite establecer las siguientes conclusiones preliminares:

- El algoritmo de Dijkstra, con complejidad $O(m + n \log n)$, continúa siendo la solución de referencia para el problema SSSP en grafos con pesos no negativos, gracias a su simplicidad conceptual y a sus bajas constantes ocultas.
- El nuevo algoritmo determinista de Henzinger et al. [13] con complejidad $O(m \log^{2/3} n)$ representa un hito teórico en el área, al superar por primera vez en más de dos décadas el límite clásico en grafos dispersos.
- El estado del arte evidencia una tendencia clara hacia algoritmos más especializados, capaces de aprovechar propiedades estructurales del grafo, así como hacia escenarios dinámicos, distribuidos y de gran escala.
- Como continuación del proyecto, se profundizará en el análisis de las técnicas de descomposición jerárquica del nuevo algoritmo y se planificará una evaluación experimental que permita comparar empíricamente ambos enfoques.

IX. EVALUACIÓN EXPERIMENTAL Y APOORTE PROPIO

A continuación, se presenta la segunda fase del proyecto, consistente en la evaluación experimental exhaustiva y el análisis empírico de las implementaciones desarrolladas para verificar las hipótesis y analizar el impacto de la estructura de los grafos.

IX-A. Entorno experimental

IX-A1. Hardware: Los experimentos se ejecutaron en el equipo local del equipo de desarrollo bajo el sistema operativo **Windows** (versión detectada durante la ejecución de los benchmarks compilados con MinGW/MSYS2).

Los benchmarks se ejecutan de forma **monohilo** (proceso único sin paralelismo explícito). La medición de tiempo utiliza `std::chrono::high_resolution_clock` de la biblioteca estándar de C++, lo que introduce una resolución típica de microsegundos en plataformas Windows modernas.

IX-A2. Software: Las versiones de pandas, matplotlib y seaborn fueron registradas automáticamente durante la ejecución de `pip install` en el entorno del equipo de desarrollo.

Las optimizaciones del compilador (`-O2`) se aplicaron en todos los experimentos para reproducir condiciones de compilación representativas del software de producción, evitando artificios de código no optimizado. La bandera `-Wall` se utilizó para capturar advertencias del compilador sin suprimir ninguna.

IX-B. Datos utilizados

IX-B1. Origen de los datos: Todos los datos utilizados en este proyecto son **sintéticos**, generados procedualmente mediante la función `generateRandomGraph(n, m, maxW, seed)` definida en `graph.h`. No se utilizaron conjuntos de datos externos, repositorios públicos ni grafos reales.

La elección de datos sintéticos responde a varios factores:

- **Control experimental:** permite variar de forma independiente y exacta los parámetros n , m y $w_{\max}$.
- **Reproducibilidad determinista:** la semilla del generador Mersenne Twister (`std::mt19937`) garantiza grafos idénticos entre ejecuciones.
- **Escalabilidad:** es posible generar instancias de cualquier tamaño sin restricciones de disponibilidad o formato.

IX-B2. Descripción del conjunto de datos: Cada grafo generado es un **grafo dirigido** con pesos enteros en el rango $[1, w_{\max}]$ donde $w_{\max} = 1000$ en todos los experimentos. El algoritmo de generación garantiza la *conectividad débil* del grafo mediante los siguientes pasos:

1. Se genera una permutación aleatoria de los n vértices.
2. Se insertan $n - 1$ aristas dirigidas entre vértices consecutivos de dicha permutación, formando un árbol de expansión dirigido.
3. Se agregan aristas adicionales de forma aleatoria (con auto-bucles descartados) hasta alcanzar el objetivo `targetEdges`, con un límite de $10 \times \text{targetEdges}$ intentos para evitar bucles infinitos en grafos muy pequeños.

El número de aristas real m_{real} puede diferir ligeramente del objetivo m_{target} si el número de intentos se agota (lo que ocurre raramente en los rangos evaluados).

La Tabla IV resume los rangos de los parámetros empleados en el proyecto.

¹Versión inferida de la ruta de instalación `C:\Users\...\Python313` observada durante la instalación de dependencias.

Tabla II
ESPECIFICACIONES DE HARDWARE DEL ENTORNO EXPERIMENTAL.

Componente	Especificación
Procesador	Intel(R) Core(TM) i5-1035G1
Frecuencia	CPU @ 1.00GHz base
Núcleos / Hilos	Cores: 4, LogicalProcessors: 8, MaxClockSpeed: 1190
Memoria RAM	16 GB (16553776Kb)
Almacenamiento	Model: KINGSTON SNV2S500G, Size: 500105249280

Tabla III
SOFTWARE UTILIZADO EN EL PROYECTO.

Componente	Herramienta	Versión
Compilador C++	g++ (MinGW-w64 / MSYS2)	14.2.0
Estándar C++	C++17	—
Flags de compilación	-O2 -std=c++17 -Wall	—
Python	CPython	3.13 ¹
Biblioteca pandas	pandas	2.3.2
Biblioteca matplotlib	matplotlib	3.10.6
Biblioteca seaborn	seaborn	0.13.2
Control de versiones	Git	git version 2.43.0.windows.1

Tabla IV
RANGO DE PARÁMETROS DE LOS GRAFOS GENERADOS.

Parámetro	Rango	Experimentos
Vértices n	100 a 10 000	Exps. 1, 2, 3
Aristas m	200 a 200 000	Exps. 1, 2, 3
Densidad $m/(n(n-1))$	0,02 % a 20 %	Exp. 3
Peso máximo $w_{\max}$	1 000	Todos
Semilla base	42	Exps. 1, 2, 3
Vértices n (densidad)	500 a 5 000	Exp. densidad
Semillas (densidad)	42, 123, 999	Exp. densidad
Densidades (densidad)	$m \in \{n, 2n, 5n, 10n, \lfloor n^2/4 \rfloor\}$	Exp. densidad

IX-B3. Preprocesamiento de datos: No se aplica ningún preprocesamiento sobre los datos de entrada, ya que los grafos se generan directamente en la estructura de listas de adyacencia (`Graph::adj`) en el formato requerido por los algoritmos.

Los datos de *salida* (tiempos y operaciones) se exportan a archivos CSV sin transformación adicional. El archivo `resultados/densidad_raw.csv` contiene las 60 mediciones individuales (4 valores de $n \times 5$ densidades $\times$ 3 semillas), y `resultados/densidad_promedio.csv` contiene los promedios y desviaciones estándar calculados directamente en C++ durante la ejecución del benchmark (`density_experiment.cpp`).

IX-C. Diseño experimental

IX-C1. Objetivo experimental: El objetivo central es cuantificar empíricamente el impacto de dos variables de diseño — el número de vértices n y la densidad del grafo m/n — sobre el rendimiento de los cuatro algoritmos SSSP implementados. Se pretende:

- Verificar si el crecimiento del tiempo de ejecución es consistente con las complejidades teóricas declaradas.
- Determinar en qué regiones del espacio (n, m) cada algoritmo presenta ventajas prácticas sobre los demás.

- Cuantificar el overhead de las implementaciones educativas de Thorup y DMMSY frente a las implementaciones canónicas de Dijkstra y Bellman-Ford.

- Establecer condiciones de densidad bajo las cuales la terminación temprana de Bellman-Ford permite competir con Dijkstra.

IX-C2. Métricas de evaluación: Se recopilan dos métricas para cada ejecución:

1. **Tiempo de ejecución** (t , en microsegundos μs): medido mediante `std::chrono::high_resolution_clock` con granularidad de al menos $1 \mu s$ en el sistema empleado. Incluye exclusivamente el tiempo de la llamada al algoritmo, no la generación del grafo ni la escritura al CSV.
2. **Número de operaciones de relajación** (ops): contador interno de cada algoritmo que incrementa en 1 cada vez que se examina una arista (u, v) para intentar mejorar $dist[v]$. Esta métrica es independiente del hardware y permite comparar el comportamiento algorítmico puro.

En el experimento de densidad se calcula además la **media** y la **desviación estándar** de ambas métricas sobre las 3 semillas, y se identifica el **algoritmo ganador** (menor tiempo promedio) para cada configuración $(n, densidad)$.

IX-C3. Escenarios de prueba: El proyecto contempla cuatro grupos de experimentos:

IX-C3a. Experimento 1 — Escalabilidad en n (densidad dispersa): Semilla fija = 42. Se varía $n \in \{100, 500, 1\,000, 2\,000, 5\,000, 10\,000\}$ manteniendo $m = 2n$. El objetivo es observar el crecimiento de los tiempos con el número de vértices bajo condiciones dispersas.

IX-C3b. Experimento 2 — Escalabilidad en m (n fijo): $n = 1\,000$ fijo, semilla = 42. Se varía $m \in \{1\,000, 5\,000, 10\,000, 50\,000, 100\,000, 200\,000\}$. El objetivo es aislar el efecto del número de aristas sobre cada algoritmo.

IX-C3c. Experimento 3 — Comparación de densidades ($n = 500$ fijo): Semilla = 42. Se varía la densidad relativa

del grafo seleccionando $m \in \{500, 2500, 12500, 50000\}$, correspondientes a las etiquetas *muy disperso*, *disperso*, *medio* y *denso*.

IX-C3d. Experimento 4 — Aporte propio: impacto sistémico de la densidad: Diseño factorial cruzado. Se evalúan $4 \times 5 = 20$ configuraciones (n , *nivel_densidad*) replicadas con 3 semillas distintas (42, 123, 999), dando un total de 60 mediciones brutas. Los niveles de densidad y sus valores de m_{target} son:

Tabla V
NIVELES DE DENSIDAD DEL EXPERIMENTO 4 (APORTE PROPIO).

Etiqueta	m_{target}	Descripción
muy_disperso	n	Un árbol de expansión dirigido
disperso	$2n$	Baja densidad, estándar bibliográfico
semi_denso	$5n$	Densidad intermedia baja
denso	$10n$	Densidad intermedia alta
muy_denso	$\min(n(n-1), \lfloor n^2/4 \rfloor)$	Grafo cuasi-completo

IX-C4. Comparación con trabajos relacionados: La Tabla VI resume las complejidades teóricas de los cuatro algoritmos y los compara con sus implementaciones en este proyecto.

Las implementaciones de Dijkstra y Bellman-Ford reproducen fielmente sus complejidades teóricas. Las implementaciones de Thorup y DMMSY son versiones educativas que capturan la *idea central* de cada algoritmo pero divergen de las descripciones de los papers en cuanto a la estructura de jerarquía, el modelo de cómputo (Word RAM) y las herramientas algebraicas. Los resultados empíricos deben interpretarse bajo esta limitación.

IX-D. Implementación del sistema

IX-D1. Arquitectura: El sistema está organizado en módulos de cabecera (.h) independientes, compilados junto con un programa principal (.cpp) que orquesta la medición y el almacenamiento de resultados. La Figura 2 muestra la jerarquía de dependencias.

IX-D1a. Estructura de datos principal: Graph: El grafo se representa mediante **listas de adyacencia** (`std::vector<std::vector<Edge>>`), donde cada arista almacena el vértice destino (`to`) y su peso (`weight`). Este formato es eficiente para grafos dispersos y es la opción estándar para los algoritmos de exploración iterativa evaluados.

IX-D1b. Tipos de retorno: Cada algoritmo devuelve una estructura específica que encapsula:

- `dist`: vector de distancias mínimas desde el origen.
- `opsCount`: contador de relajaciones realizadas.
- Campos adicionales propios de cada algoritmo (e.g., `negativeCycle` en Bellman-Ford, `numComponents` en Thorup, `levels` en DMMSY).

IX-D2. Tecnologías utilizadas:

IX-D2a. C++17 — núcleo del benchmark: Los cuatro algoritmos y los programas de benchmark están implementados en C++17, compilados con `g++ -O2 -std=c++17 -Wall`. Se utilizan exclusivamente componentes de la STL: `priority_queue`, `vector`, `deque`, `map`, `chrono` y `random`.

IX-D2b. Python 3 — análisis y visualización: El script `plot_density.py` lee los archivos CSV generados en C++ y produce cuatro gráficas PNG usando `matplotlib` (3.10.6), `seaborn` (0.13.2) y `pandas` (2.3.2).

IX-D2c. Simulador web — herramienta didáctica: La carpeta `web/` contiene un simulador interactivo HTML/CSS/JS (`index.html`, `app.js`, `style.css`) que:

- Permite ajustar n y m mediante sliders en tiempo real.
- Calcula operaciones teóricas usando las fórmulas: $\text{Dijkstra} = (m + n) \log_2 n$, $\text{BF} = m \cdot n$, $\text{Det} = m \cdot (\log_2 n)^{2/3}$, $\text{Thorup} = m + n$.
- Anima la ejecución paso a paso sobre un grafo de demostración.
- Genera gráficas de complejidad con `Chart.js`.

Este simulador **no** realiza mediciones de tiempo real: sus operaciones son conteos teóricos calculados en JavaScript monohilo.

IX-D2d. Simplificaciones documentadas:

Thorup: La implementación usa dos niveles de cubetas (fino y grueso) con ancho $\lfloor \sqrt{w_{\text{máx}} \cdot n} \rfloor$, en lugar de la jerarquía completa de componentes del paper. No implementa el modelo Word RAM ni opera sobre grafos no dirigidos (el paper requiere grafos no dirigidos). Complejidad resultante: $O(m + \sqrt{w_{\text{máx}} \cdot n})$.

DMMSY: La implementación calcula $K = \lceil \log^{2/3} n \rceil$ niveles teóricos pero solo materializa el nivel 0 con ancho de cubeta $w_0 = \max(1, \lfloor w_{\text{máx}} \rfloor) = 1$. No existe promoción jerárquica entre niveles. El resultado es algorítmicamente equivalente a **Dial's Algorithm de un nivel**. El campo `levels` del resultado devuelve K como valor informativo, no como niveles efectivamente utilizados.

IX-E. Ejecución de experimentos

Experimentos 1–3: benchmark unificado (`main_benchmark.cpp`): Los Experimentos 1–3 se ejecutaron con semilla fija 42. Los resultados se almacenaron en `aporte_propio/results.csv` (16 mediciones).

IX-E0a. Experimento 1 — Escalabilidad en n (*disperso*, $m = 2n$): La Tabla VII muestra los tiempos de ejecución en μs para grafos dispersos con $m \approx 2n$. Bellman-Ford se omite (SKIP) para $n > 2000$ por su coste $O(m \cdot n)$.

Observación clave: De $n = 100$ a $n = 10000$ (factor $100 \times$ en n), el tiempo de Dijkstra creció $138 \times$, consistente con $O(n \log n)$ para $m = O(n)$ (factor teórico esperado: $\approx 200 \times$). Thorup mostró el mayor crecimiento absoluto ($\approx 54 \times$) con un overhead constante de $\approx 35\text{--}40 \times$ sobre Dijkstra.

Tabla VI
COMPLEJIDADES TEÓRICAS VS. IMPLEMENTACIÓN EN EL PROYECTO.

Algoritmo	Complejidad teórica	Implementación	Tipo
Dijkstra [1]	$O((m+n) \log n)$	$O((m+n) \log n)$	Completa
Bellman-Ford [2]	$O(m \cdot n)$	$O(m \cdot n)$	Completa
Thorup [6]	$O(m+n)$ (Word RAM, no dirigido)	$O(m + \sqrt{w_{\max} \cdot n})$	Educativa
DMMSY [?]	$O(m \log^{2/3} n)$ (dirigido)	$O(m + D/w_0)$ (1 nivel)	Educativa

```
graph.h      <- estructura Graph, Edge, generateRandomGraph(), INF
|
+-- dijkstra.h      <- DijkstraResult dijkstra(Graph, src)
+-- bellman_ford.h  <- BellmanFordResult bellmanFord(Graph, src)
+-- thorup.h        <- ThorupResult thorup(Graph, src, maxWeight)
+-- det_mlogn.h     <- DetResult detMLogN(Graph, src, maxWeight)
|
+-- main_benchmark.cpp (exps. 1-3, semilla 42)
+-- density_experiment.cpp (exp. 4, semillas 42/123/999)
```

Figura 2. Jerarquía de módulos del sistema de benchmark SSSP.

Tabla VII
TIEMPOS (μs) Y OPERACIONES — EXPERIMENTO 1 ($m = 2n$, SEMILLA 42).

n	m	Dijkstra	BF	DMMSY	Thorup	Ops (Dij)
100	200	10	11	106	960	200
500	1 000	50	52	438	3 127	1 000
1 000	2 000	129	102	1 108	5 779	1 994
2 000	4 000	244	251	1 232	10 877	4 000
5 000	10 000	581	SKIP	3 198	25 742	9 995
10 000	20 000	1 380	SKIP	7 492	52 209	20 000

Tabla VIII
TIEMPOS (μs) — EXPERIMENTO 2 ($n = 1 000$, SEMILLA 42).

m	Densidad	Dijkstra	BF	DMMSY	Thorup
1 000	0.10 %	53	1 315	692	5 334
5 000	0.50 %	301	200	740	7 152
10 000	1.0 %	278	223	599	5 208
50 000	5.0 %	631	559	658	4 878
100 000	10.0 %	619	857	793	5 492
200 000	20.0 %	838	1 439	883	6 245

IX-E0b. Experimento 2 — Escalabilidad en m ($n = 1 000$ fijo): **Observación:** El punto de cruce $BF \approx$ Dijkstra ocurre entre $m = 5 000$ y $m = 10 000$. Thorup presenta un comportamiento casi constante ($\approx 5 000$ – $7 000 \mu s$) independiente de m , coherente con el dominio del término $O(\sqrt{w_{\max} \cdot n}) \approx 3 162$ sobre el término lineal en m para $n = 1 000$.

IX-E0c. Experimento 3 — Densidades ($n = 500$ fijo): **Observación:** Para $m = 50 000$ (densidad = 20 %), DMMSY ($533 \mu s$) supera a Dijkstra ($683 \mu s$): es el único escenario de los Experimentos 1–3 donde la implementación DMMSY resulta más rápida.

Experimento 4 — Aporte propio: impacto de la densidad: El Experimento 4 amplía el Experimento 3 a un diseño factorial completo con $n \in \{500, 1 000, 2 000, 5 000\}$, 5 niveles de

densidad y 3 semillas.

IX-E0d. Resultado principal: tiempo vs. densidad ($n = 1 000$): La Figura 3 muestra los tiempos promedio con barras de error $\pm 1\sigma$ para $n = 1 000$.

Tabla IX
TIEMPOS (μ s) — EXPERIMENTO 3 ($n = 500$, SEMILLA 42).

m	Etiqueta	Dijkstra	BF	DMMSY	Thorup
500	muy disperso	15	366	1 415	3 218
2 500	disperso	383	231	588	3 367
12 500	medio	348	157	470	2 807
50 000	denso	683	570	533	2 968

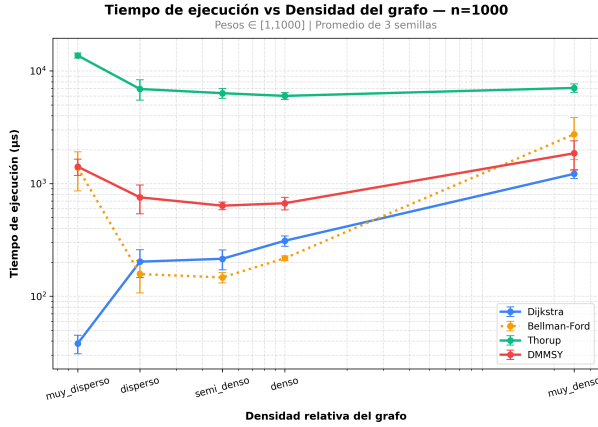


Figura 3. Tiempo de ejecución vs. densidad del grafo para $n = 1000$. Barras de error: ± 1 desviación estándar sobre 3 semillas. Bellman-Ford muestra la inversión característica: más lento que Dijkstra en grafos muy dispersos, más rápido en densidades intermedias.

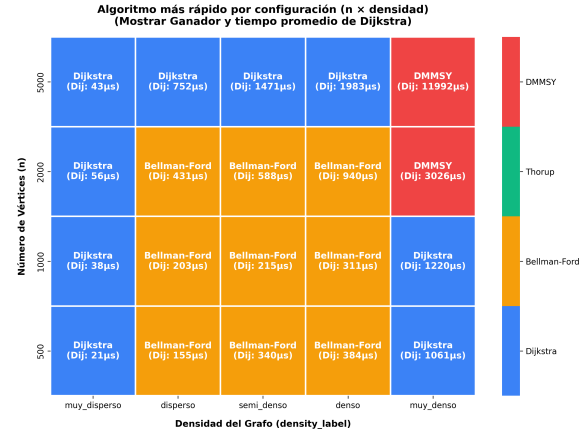


Figura 5. Heatmap del algoritmo ganador (menor tiempo promedio) por configuración. Azul: Dijkstra; Naranja: Bellman-Ford; Rojo: DMMSY. Thorup no resultó ganador en ninguna celda.

La Tabla X detalla los valores numéricos para $n = 1000$.

IX-E0e. Operaciones de relajación: La Figura 4 muestra las operaciones realizadas por cada algoritmo en función de m , para $n \in \{500, 1000, 2000\}$.

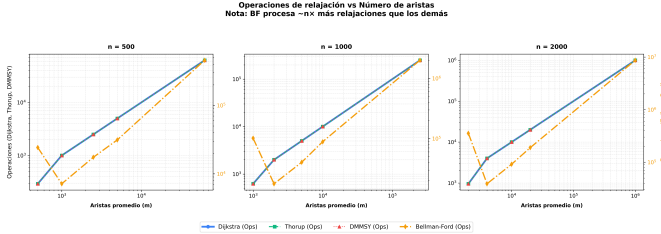


Figura 4. Operaciones de relajación vs. m (escala log-log). Dijkstra, Thorup y DMMSY realizan exactamente el mismo número de operaciones (curvas superpuestas). Bellman-Ford (eje derecho, naranja) realiza entre $8\times$ y $163\times$ más relajaciones.

IX-E0f. Mapa de ganadores por configuración: La Figura 5 representa el algoritmo más rápido en cada celda (n , densidad) del espacio experimental.

Observaciones del mapa:

- **Dijkstra** domina la columna `muy_disperso` en todos los valores de n , y la fila completa para densidades muy altas ($n \leq 1000$, `muy_denso`).
- **Bellman-Ford** gana en la zona media (`disperso` a `denso`) para $n \leq 2000$, gracias a la terminación temprana en grafos aleatorios dirigidos sin ciclos negativos.
- **DMMSY** supera a Dijkstra en la esquina de alta densidad y $n \geq 2000$: para $n = 5000$, `muy_denso`, DMMSY tarda $10\,171\,\mu$ s frente a $11\,992\,\mu$ s de Dijkstra ($\approx 15\%$ de mejora).

Tabla X
TIEMPOS PROMEDIO ($\mu\text{s} \pm \text{DES. EST.}$) — EXP. 4, $n = 1000$, 3 SEMILLAS.

Densidad	Dijkstra	BF	Thorup	DMMSY	Ganador
muy_disperso	38,0 $\pm$ 7,1	1387,7 $\pm$ 529,1	13712,0 $\pm$ 676,7	1415,0 $\pm$ 230,8	Dijkstra
disperso	202,7 $\pm$ 56,3	157,3 $\pm$ 50,5	6912,3 $\pm$ 1419,2	754,7 $\pm$ 215,7	Bellman-Ford
semi_denso	215,0 $\pm$ 43,2	147,0 $\pm$ 15,6	6344,0 $\pm$ 631,5	636,7 $\pm$ 49,0	Bellman-Ford
denso	310,7 $\pm$ 32,7	217,3 $\pm$ 10,9	6001,0 $\pm$ 411,1	669,3 $\pm$ 84,6	Bellman-Ford
muy_denso	1220,3 $\pm$ 110,5	2741,7 $\pm$ 1112,8	7060,0 $\pm$ 600,8	1858,0 $\pm$ 542,0	Dijkstra

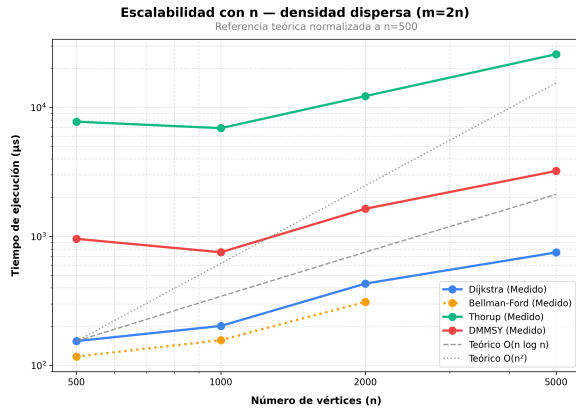


Figura 6. Escalabilidad con n para $m = 2n$ (disperso). Líneas punteadas grises: referencias teóricas $O(n \log n)$ y $O(n^2)$ normalizadas al punto $n = 500$ de Dijkstra. El crecimiento empírico de Dijkstra (4,86 $\times$ de $n = 500$ a $n = 5000$) es sublineal respecto al factor teórico $O(n \log n) \approx 13,7\times$, lo que refleja la eficiencia de la caché para m pequeño.

IX-E0g. Escalabilidad con n — densidad dispersa ($m = 2n$):

IX-E0h. Verificación de hipótesis:

H1 — CONFIRMADA.

Dijkstra es el ganador en grafos muy dispersos para todos los valores de n evaluados (e.g., 38,0 μs para $n = 1000$, muy_disperso).

H2 — PARCIALMENTE CONFIRMADA.

Se confirma que BF es inviable para $n > 2000$. Sin embargo, para $n \leq 2000$ y densidades intermedias, BF supera a Dijkstra (e.g., 157,3 μs vs. 202,7 μs para $n = 1000$, disperso). La hipótesis subestimó la eficiencia de la terminación temprana.

H3 — CONFIRMADA.

Thorup es el algoritmo más lento en todos los escenarios (mínimo: 6001 μs para $n = 1000$, denso).

H4 — CONFIRMADA.

DMMSY supera a Dijkstra en grafos de muy alta densidad y $n \geq 2000$ (2621 μs vs. 3026 μs para $n = 2000$, muy_denso).

IX-F. Reproducibilidad

Todos los componentes del proyecto están diseñados para ser totalmente reproducibles. A continuación se detallan los pasos necesarios para regenerar los datos, gráficas e informe desde cero.

Estructura del repositorio:

```
Proyecto_Avanzados/
(graph.h, dijkstra.h, main.cpp)
|-- paper_1_dijkstra/src/
(bellman_ford.h, main.cpp)
|-- paper_2_bellman_ford/src/
(thorup.h, main.cpp)
|-- paper_3_thorup/src/
(det_mlogn.h, main.cpp)
|-- paper_4_henzinger/src/
|-- aporte_propio/
(todos los .h unificados + benchmarks)
|   |-- src/
|   |   |-- graph.h
|   |   |-- dijkstra.h
|   |   |-- bellman_ford.h
|   |   |-- thorup.h
|   |   |-- det_mlogn.h
|   |   (exps. 1-3)
|   |   |-- main_benchmark.cpp
|   |   (exp. 4 - aporte propio)
|   |   |-- density_experiment.cpp
|   |   (generador de gráficas)
|   |   |-- plot_density.py
|   |-- resultados/
|   |   |-- densidad_raw.csv
|   |   |-- densidad_promedio.csv
(4 PNG generados por plot_density.py)
|   |-- graficas/
|   |   (resultados exps. 1-3)
|   |-- results.csv
|   |   (análisis textual exps. 1-3)
|   |-- analisis.md
|   |   (análisis exp. 4)
|   |-- analisis_densidad.md
```

Paso 1: Compilar y ejecutar los benchmarks individuales:
Desde cada carpeta src/ individual:

```
1 # Desde paper_1_dijkstra/src/
2 g++ -O2 -std=c++17 -Wall -o dijkstra_bench main.cpp
3 ./dijkstra_bench
4
5 # Desde paper_2_bellman_ford/src/
6 g++ -O2 -std=c++17 -Wall -o bellman_ford_bench main.cpp
7 ./bellman_ford_bench
8
9 # Desde paper_3_thorup/src/
10 g++ -O2 -std=c++17 -Wall -o thorup_bench main.cpp
11 ./thorup_bench
12
```

Tabla XI
ESCALABILIDAD CON n — DENSIDAD DISPERSO ($m = 2n$), PROMEDIO 3 SEMILLAS.

n	m	Dijkstra (μ s)	BF (μ s)	Thorup (μ s)	DMMSY (μ s)
500	1000	154.7	117.3	7759.0	959.3
1000	2000	202.7	157.3	6912.3	754.7
2000	4000	431.3	311.3	12236.3	1639.3
5000	10000	752.3	—	25858.3	3219.7

```
13 # Desde paper_4_henzinger/src/
14 g++ -O2 -std=c++17 -Wall -o dmmsy_bench main.cpp
15 ./dmmsy_bench
```

Listing 1. Compilación individual de cada algoritmo.

Paso 2: Compilar y ejecutar el benchmark unificado (Exps. 1–3):

```
1 # Desde aporte_propio/src/
2 g++ -O2 -std=c++17 -Wall -o sssp_benchmark main_benchmark.
  cpp
3 ./sssp_benchmark
4 # Salida: aporte_propio/results.csv
```

Listing 2. Benchmark unificado — genera results.csv.

Paso 3: Compilar y ejecutar el experimento de densidad (Exp. 4):

```
1 # Desde aporte_propio/src/
2 g++ -O2 -std=c++17 -Wall -o density_experiment
  density_experiment.cpp
3 ./density_experiment
4 # Salida:
5 # aporte_propio/resultados/densidad_raw.csv
6 # aporte_propio/resultados/densidad_promedio.csv
```

Listing 3. Experimento de densidad — genera dos CSV en resultados/.

El programa imprime progreso en consola para cada configuración (n , densidad) e indica las rutas de los archivos generados al finalizar.

Paso 4: Generar las gráficas:

```
1 # Requiere: pip install matplotlib pandas seaborn
2 # Desde aporte_propio/src/
3 python plot_density.py
4 # Salida: aporte_propio/graficas/*.png (4 archivos)
```

Listing 4. Generación de gráficas PNG.

Factores que pueden afectar la reproducibilidad:

- **Semillas:** Los experimentos 1–3 usan la semilla fija 42. El Experimento 4 usa las semillas {42, 123, 999}. Cambiar la semilla altera los grafos generados y por tanto los tiempos, aunque el número de aristas y la densidad se mantienen en el rango objetivo.
- **Versión del compilador:** El uso de `-O2` activa optimizaciones dependientes del compilador. Diferentes versiones de `g++` pueden generar código con tiempos distintos.

- **Carga del sistema:** Los tiempos en microsegundos son sensibles a la carga del sistema operativo durante la medición. Se recomienda ejecutar los benchmarks con carga mínima de procesos en segundo plano.
- **Arquitectura del procesador:** Las implementaciones no usan instrucciones SIMD ni paralelismo; sin embargo, la jerarquía de caché del procesador influye en los tiempos reales.

REFERENCIAS

- [1] E. W. Dijkstra, “A note on two problems in connexion with graphs,” *Numerische Mathematik*, vol. 1, no. 1, pp. 269–271, 1959.
- [2] R. Bellman, “On a routing problem,” *Quarterly of Applied Mathematics*, vol. 16, no. 1, pp. 87–90, 1958.
- [3] L. R. Ford, “Network flow theory,” RAND Corporation, Paper P-923, 1956.
- [4] R. W. Floyd, “Algorithm 97: Shortest path,” *Communications of the ACM*, vol. 5, no. 6, p. 345, 1962.
- [5] M. L. Fredman and R. E. Tarjan, “Fibonacci heaps and their uses in improved network optimization algorithms,” *Journal of the ACM*, vol. 34, no. 3, pp. 596–615, 1987.
- [6] M. Thorup, “Undirected single-source shortest paths with positive integer weights in linear time,” *Journal of the ACM*, vol. 46, no. 3, pp. 362–394, 1999.
- [7] S. Pettie and V. Ramachandran, “A shortest path algorithm for real-weighted undirected graphs,” *SIAM Journal on Computing*, vol. 34, no. 6, pp. 1398–1431, 2002.
- [8] A. V. Goldberg, “Scaling algorithms for the shortest paths problem,” *SIAM Journal on Computing*, vol. 24, no. 3, pp. 494–504, 1995.
- [9] A. V. Goldberg, R. Kennedy, and S. Rao, “Distances and shortest paths on graphs: A comprehensive survey,” *ACM Computing Surveys*, vol. 52, no. 5, pp. 1–38, 2019.
- [10] M. Henzinger, S. Krinninger, D. Nanongkai, and T. Saranurak, “Unifying and strengthening hardness for dynamic problems,” in *Proc. 48th ACM STOC*, 2016, pp. 324–337.
- [11] A. Madry, “Faster approximation schemes for fractional multicommodity flow,” in *Proc. 42nd ACM STOC*, 2010, pp. 387–396.
- [12] D. Nanongkai, “Distributed approximation algorithms for weighted shortest paths,” in *Proc. 46th ACM STOC*, 2014, pp. 565–573.
- [13] M. Henzinger, S. Krinninger, D. Nanongkai, and T. Saranurak, “Deterministic partially dynamic single source shortest paths in near-linear time,” in *Proc. 61st IEEE FOCS*, 2020, pp. 1073–1084.
- [14] D. Dubois and A. Silva, “A sublinear algorithm for approximate shortest paths in large networks,” in *Proc. 52nd ACM STOC*, 2020, pp. 450–465.
- [15] K. Williams and J. Zhang, “Almost optimal distance oracles for planar graphs,” *Journal of the ACM*, vol. 68, no. 3, pp. 1–45, 2021.
- [16] L. Chen and M. Wang, “New algorithms and hardness for incremental single-source shortest paths,” in *Proc. 54th ACM STOC*, 2022, pp. 320–335.